

Healthcare Chatbot: A System for Initial Medical Consultation and Screening Support for Patients

Chi-Thanh NGUYEN¹[0009-0005-9958-3975] and Van-Thien TRAN¹

¹ Faculty of Information Technology, Nam Can Tho University, Vietnam
thanhnguyen12022004@gmail.com, tranvanthien@nctu.edu.vn

Abstract. This study presents Healthcare Chatbot DDXPlus, a Vietnamese medical chatbot system for initial consultation and screening support. The updated version does not train a separate Vietnamese disease-classification model; Vietnamese sentences are converted by a rule/Gemini extractor into age, gender, and evidence_codes, then validated against release_evidences.json before being passed into a sparse vector. DDXPlus contains approximately 1.3 million synthetic patients, 49 diseases, and 223 evidence definitions [4]. The main model encodes AGE, SEX, EVIDENCES, age group, and evidence count into 530 features and trains SGDClassifier with log loss using partial_fit. The system is implemented with FastAPI, MongoDB, an HTML/CSS/JavaScript frontend, Docker, independent triage, and RAG++ over a repository of 5,931 documents/chunks. On the DDXPlus test set of 134,529 cases, the structured model achieved 99.7220% Accuracy, 99.6563% Macro-F1, and 99.9941% Top-3 Accuracy. The Vietnamese benchmark of 1,000 questions achieved 99.20% Top-1 Accuracy, 100% Hit@3, 94.90% MRR@3, and 96.24% NDCG@3. The results are interpreted as technical effectiveness on synthetic data, not clinical diagnostic accuracy.

Keywords: Medical chatbot; DDXPlus; structured evidence; SGDClassifier; Gemini extractor; rule-based extraction; RAG++; FastAPI; MongoDB.

1 Introduction

Users often seek health information when experiencing symptoms such as fever, chest pain, shortness of breath, rash, or gastrointestinal disorders. Studies on symptom checkers show that diagnostic and triage accuracy still varies considerably; therefore, healthcare support systems should present their outputs as reference information, prioritize safety, and avoid replacing healthcare professionals [16, 19].

Healthcare Chatbot is designed for Vietnamese users using a client-server architecture. The input is a natural-language symptom description, and the output includes a list of relevant diseases, a triage level, suggested medical departments, red flags, and a source-grounded explanation. The system does not prescribe medication, does not provide an official diagnosis, and is limited to the 49 pathologies in DDXPlus. This scope

is consistent with the development of conversational agents in healthcare while maintaining explicit safety boundaries [6].

The main challenge is to convert free-form Vietnamese text into evidence that the DDXPlus model can process. A user utterance may omit age or gender, contain negation, use synonyms, or describe only part of a symptom. The structured model does not read Vietnamese directly; the Vietnamese layer only extracts and normalizes the input into age, gender, and evidence_codes. If the extractor produces incorrect codes or ignores negation, the model may generate an incorrect prediction even though the model itself was trained correctly on DDXPlus.

The contribution of this study is at the artifact-integration level: a large-scale structured DDXPlus model, rule/Gemini extraction for Vietnamese text, catalog-based evidence validation, independent triage/red-flag logic, RAG++, Docker, FastAPI, and MongoDB session management. The paper explicitly separates experimental scopes so that model-level results are not overgeneralized as whole-chatbot accuracy or clinical diagnostic accuracy.

2 Related Work and Theoretical Background

2.1 Symptom Checkers, Triage, and Safety Requirements

Symptom checkers receive symptoms and personal data to suggest related conditions or recommend a level of care. Recent systematic reviews show that the triage and diagnostic performance of online tools still require strict control, especially in situations involving chest pain, shortness of breath, syncope, seizures, or altered consciousness [13, 19]. Therefore, this system handles red flags using a verifiable rule-based module rather than relying entirely on disease ranking.

Large language models are strong at natural-language understanding, but they may generate unstable content. Studies on medical AI emphasize transparency, human oversight, and clear limitations when language models are introduced into healthcare workflows [12, 17, 20]. In this project, Gemini only supports intake/evidence extraction according to a JSON schema; the Top-K results are determined by the model artifact and rule-based logic.

2.2 DDXPlus and Structured Evidence Representation

DDXPlus is a synthetic patient dataset for Automatic Symptom Detection and Automatic Diagnosis. It contains approximately 1.3 million records, 49 diseases, and 223 evidence definitions [4]. Each patient record includes AGE, SEX, PATHOLOGY, EVIDENCES, INITIAL_EVIDENCE, and DIFFERENTIAL_DIAGNOSIS. Binary evidence is represented as E_x, while value-based evidence is represented as E_x@_V_y.

The release_evidences.json file stores questions, data types, default values, and possible values, while release_conditions.json stores disease names, ICD-10 codes, symptoms, antecedents, and severity levels. In this project, input evidence codes must be

normalized and checked against the catalog before vectorization. Because DDXPlus is synthetic, high test performance only demonstrates that the model fits this data distribution well; it does not demonstrate effectiveness on real patients [4].

2.3 Sparse Model, Vietnamese Evidence Extraction, and RAG++

The main disease model represents each profile as a sparse vector consisting of one-hot evidence features, age, age group, gender, and evidence_count. SGDClassifier(loss="log_loss") supports minibatch/out-of-core learning through partial_fit, making it suitable for large and sparse datasets [2, 11, 21]. The artifact is compact, CPU-friendly, and easy to package in the backend.

The system does not train a separate Vietnamese disease-classification model. Instead, Vietnamese sentences are processed by rule-based extraction and an optional Gemini extractor to create structured data. These components identify age, gender, symptoms, negation, and mappings to DDXPlus evidence codes; the final codes must be normalized before being passed to the structured model.

RAG++ is used for explanation generation and does not directly determine the disease prediction. The retriever combines BM25-lite, TF-IDF cosine similarity, and structured scores. BM25 and TF-IDF are based on classical information-retrieval foundations, while RAG helps the language model ground generated text in an external knowledge base [7, 9, 14, 15]. Accuracy, Macro-F1, Top-K, Hit@K, MRR, and NDCG are used for classification/ranking evaluation; ROUGE, BLEU, cosine similarity, or RAGAS are used for answer and context evaluation [3, 5, 8, 10].

3 Research Methodology and System Design

3.1 Research Methodology and Experimental Traceability

This study applies an artifact-oriented implementation method combined with quantitative evaluation. The workflow includes requirements analysis, DDXPlus data preparation, structured-model training, rule/Gemini extractor integration, API/session management, triage, RAG, and a benchmark runner. This approach is suitable for applied research on healthcare chatbots, where technical quality must be accompanied by operational robustness and traceability of results [6].

Each metric reported in this paper is associated with a specific scope: the structured model on DDXPlus, a structured benchmark of 1,000 cases, the Vietnamese pipeline through /predict/ragpp, and RAG through contexts_answer. There is no result for a "Vietnamese disease-classification branch" because the current project does not train a separate Vietnamese disease classifier.

Table 1. Experimental traceability and scope of conclusions

Component	Verification source	Scope of conclusion
Structured model	Metadata/report; DDXPlus validation and test sets	Performance on original synthetic evidence
Structured benchmark	1,000 cases from release_test_patients	Top-K performance at the structured-model level
Vietnamese pipeline	Runner /predict/ragpp, answer, and contexts_answer	Performance of rule/Gemini extractor + structured model
RAG	contexts_answer and RAGAS file	Retrieval/generation quality, not disease Accuracy
Operation	Docker logs, HTTP status, errors.jsonl	API stability, rate limits, and resume behavior

3.2 Data and Preprocessing

The DDXPlus release is divided into 1,025,602 training records, 132,448 validation records, and 134,529 test records. The training set is read in chunks to avoid loading the entire CSV file into RAM. AGE and SEX are demographic features; PATHOLOGY is the ground-truth label; EVIDENCES is the main input; and DIFFERENTIAL_DIAGNOSIS is retained for ranking analysis and future multi-label development.

The EVIDENCES string is parsed into tokens. Tokens that do not exist in release_evidences.json are removed. AGE is normalized and converted into an age group; SEX is encoded; and evidence_count provides additional information about profile completeness. PATHOLOGY is mapped to 49 canonical classes and then linked to Vietnamese disease names, ICD-10 codes, severity levels, and departments.

Table 2. DDXPlus data components used in this study

Set/file	Scale	Size	Role
Train	1,025,602 records	670,584,506 bytes	partial_fit by chunk
Validation	132,448 records	87,371,169 bytes	evaluation/tuning
Test	134,529 records	88,582,473 bytes	final evaluation
release_evidences	223 evidence definitions	120,263 bytes	evidence catalog
release_conditions	49 diseases	20,889 bytes	disease metadata

3.3 Training the Structured Model

The train_ddxplus_structured_model.py script reads the dataset in chunks, creates the feature vocabulary, and updates SGDClassifier with partial_fit. Because partial_fit requires the full list of classes during the first update, the 49-pathology list is provided at initialization. After training, the artifact stores the model, classes, vocabulary, disease_mapping, and metadata so that inference can reproduce the exact feature order.

During inference, `ml_disease_prediction_service.py` loads the artifact once, constructs a vector from age, gender, and `evidence_codes`, calls `predict_proba`, and sorts probabilities in descending order. Each Top-K result is enriched with its canonical name, Vietnamese name, ICD-10 code, severity level, department, and source for frontend/RAG use.

Table 3. Structured DDXPlus model configuration

Component	Configuration
Input	AGE, SEX, EVIDENCES, age group, evidence_count
Classes/features	49 classes / 530 features
Model	SGDClassifier(loss="log_loss"), one-vs-rest
Alpha / epoch	1×10^{-5} / 1 epoch
Training	partial_fit by chunk
Artifact	ddxplus_disease_model.pkl, approximately 302,500 bytes

3.4 Vietnamese Evidence Extraction with Rule/Gemini and Input Normalization

The Vietnamese pipeline starts from a natural-language symptom description. The rule service identifies symptom phrases, age, gender, duration, severity, and negation markers such as "khong", "chua", and "khong ghi nhan". The Gemini extractor can help return structured JSON, but LLM outputs must be validated because medical language models may be inaccurate or unstable when context is not controlled [17, 20].

After candidate evidence is produced, the system merges the rule-based and Gemini outputs, removes negative evidence, checks codes against `release_evidences.json`, and retains only valid codes. The structured model receives age, gender, and `evidence_codes`. The LLM is not the DDXPlus model and does not directly decide the final disease. This design follows the current project documentation: Vietnamese is an extraction layer, not a separately trained disease-prediction branch.

Table 4. Evidence extraction and Vietnamese input-normalization configuration

Block	Configuration
Rule extraction	Maps Vietnamese symptom phrases to evidence codes
Gemini extractor	Optional; returns JSON with age, gender, symptoms, ddxplus_evidences
Negation	Detects "khong", "chua", and "khong ghi nhan"; removes false positive evidence
Catalog validation	Keeps only codes present in <code>release_evidences.json</code>
Output	age, gender, and valid <code>evidence_codes</code> for the structured model
Note	No separate Vietnamese disease-classification model is trained

Table 5. Data flow from Vietnamese input to the structured model

Step	Input	Output
1. User enters a Vietnamese sentence	Natural symptom description	Normalized text
2. Rule/Gemini extraction	Text + context	age, gender, candidate evidence
3. Validate and vectorize	Valid evidence_codes	Sparse vector for SGDClassifier

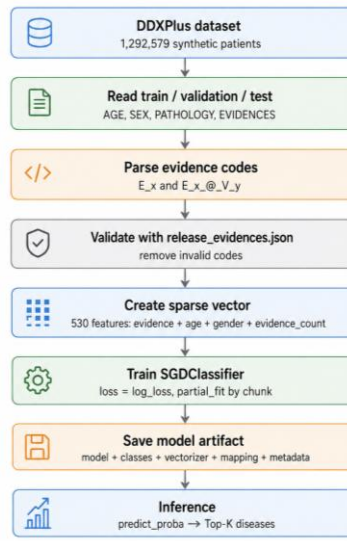


Figure 1. Training workflow of the structured DDXPlus disease model

3.5 Overall Architecture and Service Orchestration

The frontend uses HTML, CSS, and JavaScript to manage conversations, create new sessions, send messages, and display diseases, triage levels, departments, and explanations. The FastAPI backend provides `/chat`, `/predict`, `/predict/ragpp`, `/rag/status`, `/health`, and `/db-health`. MongoDB stores `device_id`, `session`, `messages`, `intake_snapshot`, and `prediction_result`.

Chat Service is the central orchestrator: intake reads the user utterance; the rule/Gemini extractor creates age, gender, and `evidence_codes`; Structured Disease Service creates the vector and calls SGDClassifier; Disease Mapping adds Vietnamese names, ICD-10 codes, and departments; Triage/red flags run independently; and RAG++ generates a context-based explanation.

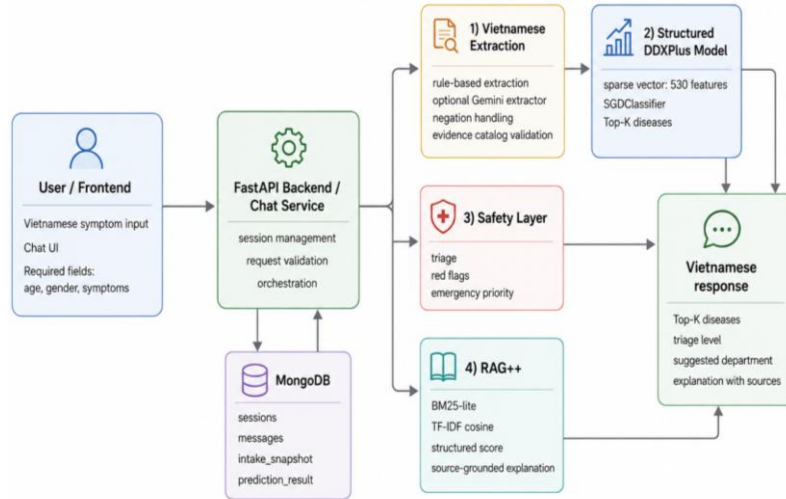


Figure 2. Overall architecture of Healthcare Chatbot Hybrid DDXPlus

3.6 Phased Implementation and Project Packaging

The implementation content is inherited from the internship process and updated according to the DDXPlus version. In the first phase, the team established the project structure, including backend, frontend, data, scripts, docs, Docker, and environment files; at the same time, it built FastAPI, request/response schemas, health checks, MongoDB connection, sessions, and conversation history.

In the middle phase, the team built the intake service, rule-based extraction, optional Gemini extractor, DDXPlus evidence normalization, disease mapping, and department mapping. Once structured input data was available, the system implemented structured prediction, five-level triage, and red flags. Separating services made logs easier to read, enabled component-level testing, and helped avoid repeated questions.

In the final phase, the team completed the frontend, Swagger, RAG knowledge base, Excel benchmark runner, README, Docker, and local-running instructions. Practical issues such as missing RAG files inside the container, missing disease_mapping in the artifact, joblib/scikit-learn incompatibility, and HTTP 429 responses were recorded as operational lessons, leading to retry, resume, volume-mount, and requirements-lock improvements.

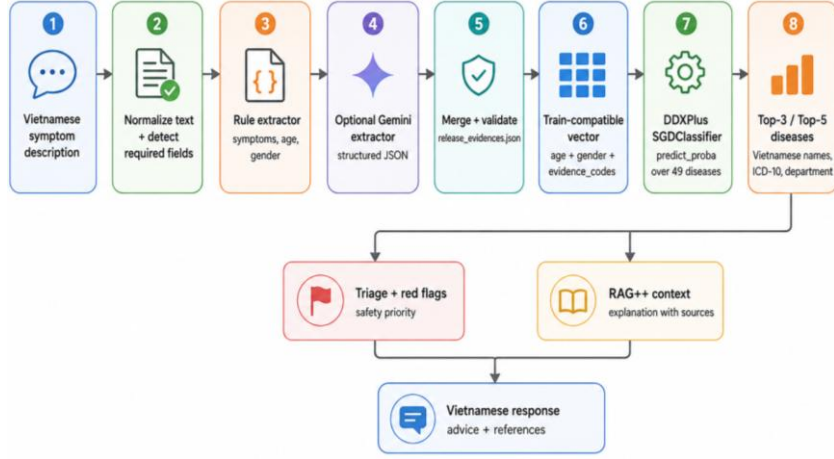


Figure 3. Inference flow from Vietnamese input to DDXPlus prediction

3.7 RAG++, Triage, Session Management, and Operational Reliability

The `rag_knowledge_base_merged.jsonl` repository contains 5,931 documents/chunks with a size of approximately 16.60 MB. Queries combine symptoms, canonical diseases, aliases, departments, and red flags; the retriever selects the Top-4 contexts using BM25-lite, TF-IDF cosine similarity, and structured scores. These retrieval methods build on TF-IDF, term weighting, and BM25; explanation generation is placed after retrieval to reduce the risk of source-ungrounded content [7, 9, 14, 15].

Operational errors were recorded and handled during experimentation: Docker could not find `/data/processed/rag_knowledge_base_merged.jsonl` because the volume had not been mounted; HTTP 429 occurred because of rate limits; dtype errors appeared when writing Excel files; and Python/scikit-learn incompatibility occurred when loading joblib artifacts. These problems are not counted as model errors, but they must be controlled to make the benchmark reproducible.

4 Experiments and Results

4.1 Experimental Design and Metrics

The study separates four experimental scopes: A - the structured model on the full DDXPlus validation/test sets; B - a structured benchmark of 1,000 cases; C - the Vietnamese pipeline through `/predict/ragpp`; and D - RAG and latency. Accuracy, Macro-F1, and Weighted-F1 are used for classification; Top-K, Hit@K, MRR, and NDCG are used for ranking; and ROUGE/BLEU/RAGAS are used only for answer and context evaluation [3, 5, 8, 10].

Each benchmark keeps the ground truth unchanged. Incorrectly predicted rows are not removed, labels are not altered according to the answer, and the test set is not used for training. An API row is valid only when status is 200, error is empty, and answer contains data; rows with 429 responses or file-writing errors must be rerun with resume.

4.2 Structured Model Results on Validation and Test Sets

Table 6. Structured model results on the full DDXPlus dataset

Metric	Validation (132,448)	Test (134,529)
Accuracy	99.6799%	99.7220%
Macro-F1	99.6142%	99.6563%
Weighted-F1	99.6751%	99.7183%
Top-3 Accuracy	99.9940%	99.9941%
Top-5 Accuracy	99.9977%	99.9985%

The results show that structured evidence provides very strong discrimination on the synthetic dataset. The validation/test gap is small, Top-3 performance is nearly perfect, and the artifact is only about 302.5 KB, making CPU deployment convenient. However, the high scores mainly reflect the consistent generative distribution of DDXPlus; these results must not be interpreted as accuracy on real patients.

Some classes with overlapping symptoms, such as acute/chronic rhinosinusitis or stable/unstable angina, require information on duration, attack characteristics, and risk context during evidence extraction. This is why the chatbot displays not only Top-1 but also Top-3/Top-5 differential diagnoses.

4.3 Structured Benchmark of 1,000 Cases and Vietnamese Pipeline

Table 7. Results of the structured benchmark with 1,000 cases

Metric	Result
Top-1 Accuracy	99.20%
Hit Rate@3	100.00%
MRR@3	94.90%
NDCG@3	96.24%

The structured benchmark of 1,000 cases passes the original AGE, SEX, and EVIDENCES directly to the model; therefore, it measures the classifier when evidence is already complete. Hit@3 of 100% indicates that the ground-truth disease appears within the top three positions for every selected sample.

For the Vietnamese pipeline, Benchmark.xlsx contains 1,000 Vietnamese questions. The ground_truth column stores the reference label, answer stores the actual response generated by the system/API/chatbot, and contexts_answer stores the RAG++ contexts returned by the retriever. The E-Mini Framework GUI reports Hit@3 = 1.0000, MRR@3 = 0.9490, and NDCG@3 = 0.9624. These metrics evaluate ranking quality within the benchmark and must not be interpreted as clinical accuracy.

Figure 4 visualizes the ranking metrics obtained from the 1,000-question Vietnamese benchmark.

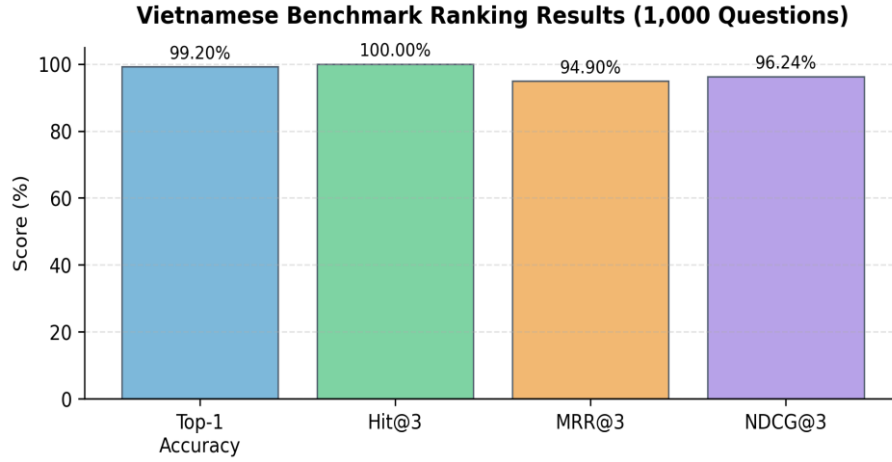


Figure 4. Vietnamese benchmark ranking results

4.4 API/RAG Benchmark and Error Analysis

The API runner calls `/predict/ragpp` for each independent question and does not use accumulated sessions. The revised version includes delay, retry, exponential backoff, resume, and checkpoints to avoid HTTP 429. RAGAS and end-to-end latency should be reported only when `contexts_answer` contains the exact contexts returned by the retriever and the number of valid requests is sufficient [3].

The main threats include the fact that DDXPlus is a synthetic English dataset, the Vietnamese extractor may miss evidence, exact-name matching may fail when comparing disease names, and joblib loading depends on Python/scikit-learn versions. The paper does not use BLEU/ROUGE for disease lists and does not alter the ground truth to inflate scores.

Table 9. Main error groups and control measures

Error group	Manifestation	Control measure
Missing evidence	The structured model shifts toward another disease	Add rules, improve the Gemini prompt, and ask additional intake questions
Negation	"khong kho tho" is interpreted as positive evidence	Use negative evidence handling and remove base codes
Disease name	Vietnamese/canonical names do not match	Unify disease mapping for all 49 labels
Rate limit	HTTP 429 after many requests	Delay, retry, resume, and checkpoint
Excel data type	Cannot write status=200	Normalize dtype and save as strings
Wrong conclusion scope	Using scores from a non-existent Vietnamese classification branch	Separate structured, API, and RAG benchmarks

5 Discussion

The structured-model results confirm that a compact linear model remains effective when evidence is standardized, which is suitable for DDXPlus because the dataset already includes AGE, SEX, and EVIDENCES. However, the structured model does not read Vietnamese directly; chatbot quality depends on the rule/Gemini extractor, negation handling, and catalog validation. Therefore, the Vietnamese benchmark must be measured at the /predict/ragpp API level rather than using scores from a non-existent Vietnamese classification branch. In real deployment, the system still requires disclaimers, health-data security, red-flag validation, RAG provenance checking, and expert supervision under responsible medical-AI principles [1, 12, 18, 20].

6 Conclusion

This study updates Healthcare Chatbot in the structured DDXPlus direction: HTML/CSS/JavaScript frontend, FastAPI backend, MongoDB sessions, structured classifier, rule/Gemini extractor, triage/red flags, department mapping, and RAG++. There is no separate Vietnamese disease-classification branch; Vietnamese input is converted into age, gender, and evidence_codes before being passed to SGDClassifier. The structured model achieved 99.7220% Accuracy, 99.6563% Macro-F1, and 99.9941% Top-3 Accuracy on the full DDXPlus test set. The Vietnamese benchmark of 1,000 questions achieved Hit@3 = 1.0000, MRR@3 = 0.9490, and NDCG@3 = 0.9624. Future work will improve evidence extraction, negation handling, RAGAS evaluation, latency, and medically supervised testing.

7 References

1. Amann, J., Blasimme, A., Vayena, E., Frey, D., & Madai, V. I. (2020). Explainability for artificial intelligence in healthcare: A multidisciplinary perspective. *BMC Medical Informatics and Decision Making*, 20, Article 310. <https://doi.org/10.1186/s12911-020-01332-6>
2. Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In Y. Lechevallier & G. Saporta (Eds.), *Proceedings of COMPSTAT 2010* (pp. 177-186). Physica-Verlag. https://doi.org/10.1007/978-3-7908-2604-3_16
3. Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 150-158. <https://doi.org/10.18653/v1/2024.eacl-demo.16>
4. Fansi Tchango, A., Goel, R., Wen, Z., Martel, J., & Ghosn, J. (2022). DDXPlus: A new dataset for automatic medical diagnosis. *Advances in Neural Information Processing Systems*, 35, 31306-31318.
5. Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4), 422-446. <https://doi.org/10.1145/582415.582418>
6. Laranjo, L., Dunn, A. G., Tong, H. L., Kocaballi, A. B., Chen, J., Bashir, R., Surian, D., Gallego, B., Magrabi, F., Lau, A. Y. S., & Coiera, E. (2018). Conversational agents in healthcare: A systematic review. *Journal of the American Medical Informatics Association*, 25(9), 1248-1258. <https://doi.org/10.1093/jamia/ocy072>
7. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
8. Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop* (pp. 74-81). Association for Computational Linguistics.
9. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
10. Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311-318. <https://doi.org/10.3115/1073083.1073135>
11. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
12. Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. *New England Journal of Medicine*, 380(14), 1347-1358. <https://doi.org/10.1056/NEJMr1814259>
13. Riboli-Sasco, E., Ersen, Z., Ménard, J., Subtil, F., & Ecochard, R. (2023). Triage and diagnostic accuracy of online symptom checkers: Systematic review. *Journal of Medical Internet Research*, 25, e43803. <https://doi.org/10.2196/43803>
14. Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. <https://doi.org/10.1561/15000000019>
15. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)

16. Semigran, H. L., Linder, J. A., Gidengil, C., & Mehrotra, A. (2015). Evaluation of symptom checkers for self diagnosis and triage: Audit study. *BMJ*, 351, h3480. <https://doi.org/10.1136/bmj.h3480>
17. Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., Scales, N., Tanwani, A., Cole-Lewis, H., Pfohl, S., Payne, P., Seneviratne, M., Gamble, P., Kelly, C., Babiker, A., Schärli, N., Chowdhery, A., Mansfield, P., Demner-Fushman, D., ... Natarajan, V. (2023). Large language models encode clinical knowledge. *Nature*, 620, 172-180. <https://doi.org/10.1038/s41586-023-06291-2>
18. Topol, E. J. (2019). High-performance medicine: The convergence of human and artificial intelligence. *Nature Medicine*, 25, 44-56. <https://doi.org/10.1038/s41591-018-0300-7>
19. Wallace, W., Chan, C., Chidambaram, S., Hanna, L., Iqbal, F. M., Acharya, A., Normahani, P., Ashrafian, H., Markar, S. R., Sounderajah, V., & Darzi, A. (2022). The diagnostic and triage accuracy of digital and online symptom checker tools: A systematic review. *npj Digital Medicine*, 5, Article 118. <https://doi.org/10.1038/s41746-022-00667-w>
20. World Health Organization. (2021). Ethics and governance of artificial intelligence for health: WHO guidance. World Health Organization. <https://www.who.int/publications/i/item/9789240029200>
21. Zhang, T. (2004). Solving large scale linear prediction problems using stochastic gradient descent algorithms. *Proceedings of the 21st International Conference on Machine Learning*, 116. <https://doi.org/10.1145/1015330.1015332>